Chapter 3

WEBSITE DESIGN

3.1 Sitemap

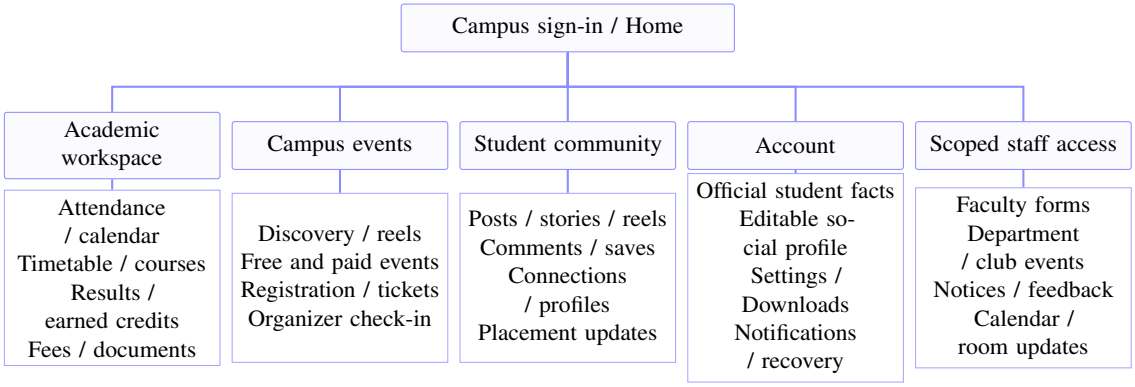


Figure 3.1: Architecture and sitemap of the project

Figure 3.1 shows the navigation structure of *Smart College Management Platform*. The shared Home workspace leads to academic records, campus events, student community and account settings. Attendance, timetable, registered courses, results, earned credits, fees and documents belong to the academic area. Events lead to registration and private tickets; community links posts, reels and connections. Faculty, organizer and administrator tools appear only within their assigned permissions. Official student details remain separate from editable social profiles. The screen captures in this chapter use the implemented project with fictional demonstration records.

3.2 Design Phase

3.2.1 Data Flow Diagram

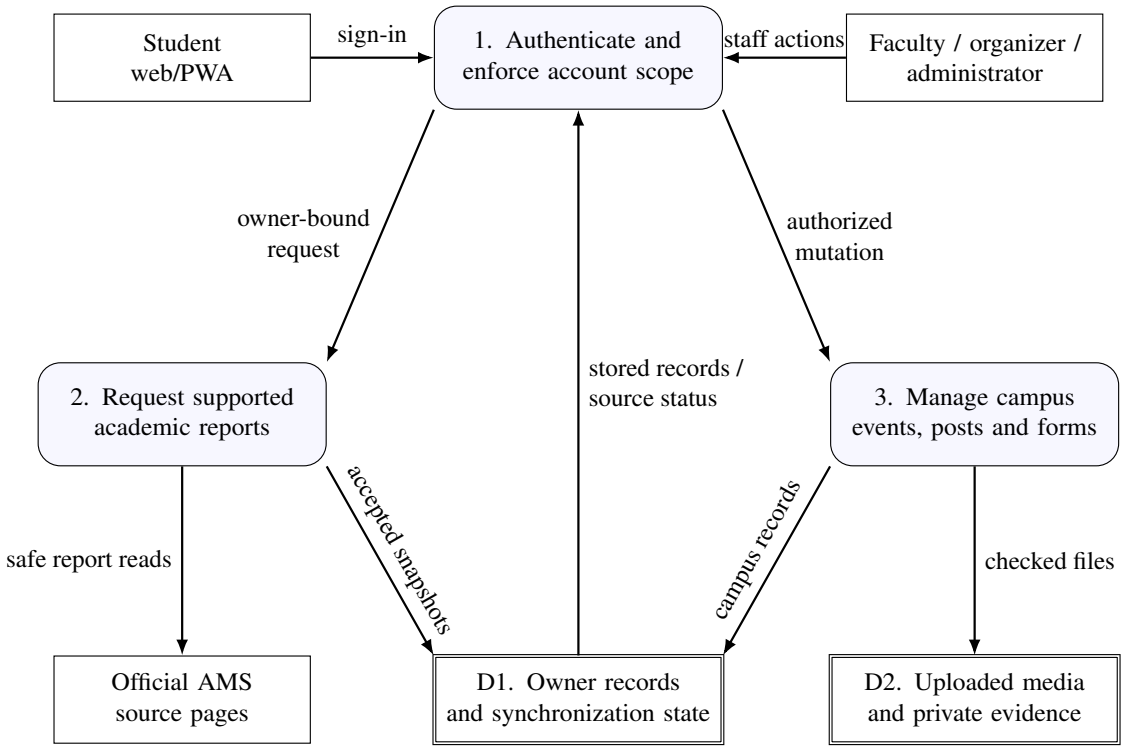


Figure 3.2: Data Flow Diagram

Figure 3.2 shows the flow of academic and campus information. Authentication establishes account ownership before protected requests are accepted. The academic adapter reads supported AMS reports and retains accepted snapshots with source timestamps. Campus services manage events, social interactions and faculty forms under role and scope checks. Uploaded media and private evidence have separate visibility rules. The browser receives the records and their synchronization status; a failed source section retains its last accepted data rather than creating missing university records.

3.3 Front End and Back End Design

Front End

React interface: academic dashboard, events and community.

Back End

Node.js campus API, Python academic service and MongoDB.

3.3.1 Home Page

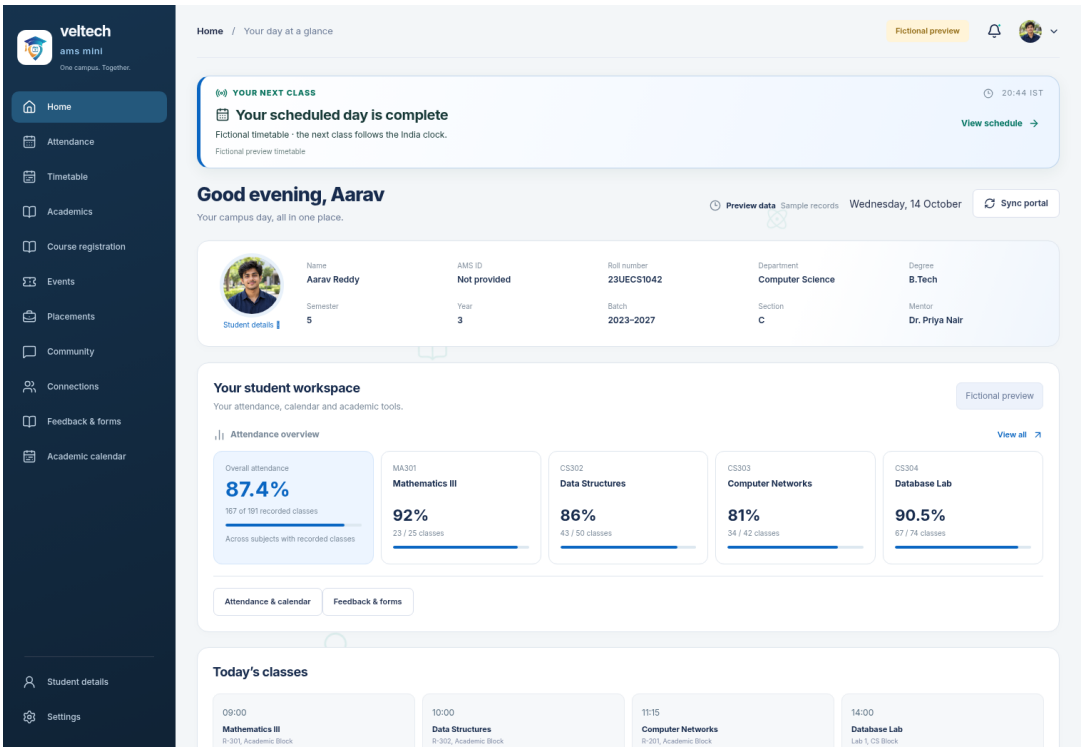


Figure 3.3: Home page of the project

Figure 3.3 shows the student Home workspace. The current or next class uses India time and the available source timetable. A compact identity panel presents available student facts and a portrait. Overall and subject attendance, academic shortcuts and notices follow. Present or absent colours use supplied attendance marks; missing information remains distinct. The demonstration screenshot illustrates the interface and does not represent an author's university record.

3.3.2 Campus Login and Session Restoration

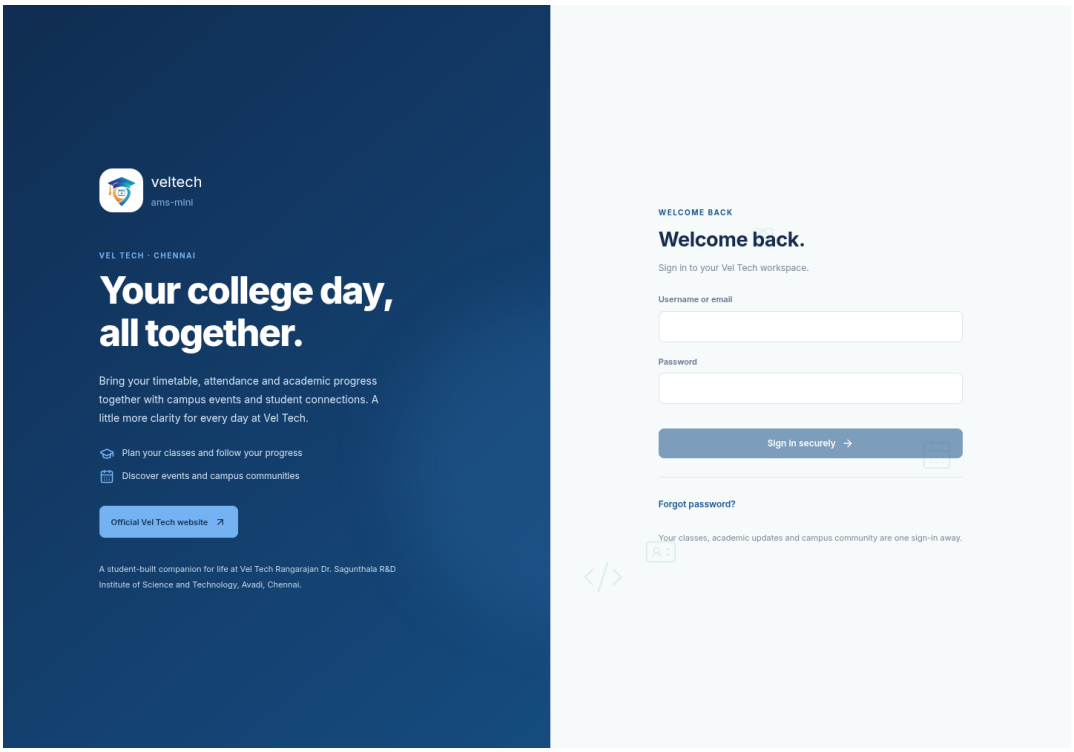


Figure 3.4: Campus login page

Figure 3.4 shows the common sign-in page. Students, faculty and administrators enter their username or email and password in the same interface; persisted account permissions determine the destination. Staff credentials remain separate from a student's AMS connection. The client restores a valid campus session before showing login and offers retry for temporary connection failures. University verification appears only when required by AMS. The forgot-password flow uses a verified address and a browser-bound, single-use code; it does not change the university password.

3.3.3 Form Validation

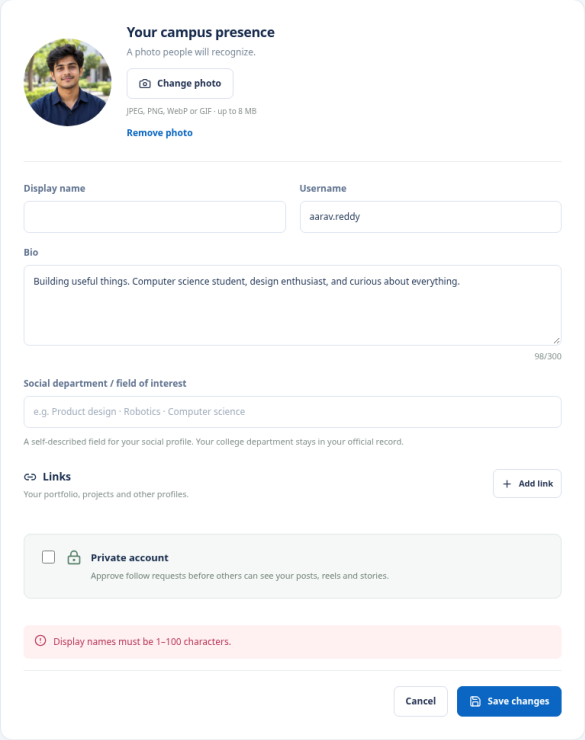


Figure 3.5: Form validation in the project

Figure 3.5 shows an actual validation response from the React social-profile form. A whitespace-only display name is rejected after trimming, and the form keeps the student on the same screen to correct it. Username format, field lengths and social links are also checked. Server-side ownership and permission checks protect saved changes. The editable social identity is separate from official college records, so changing a display name, bio or social department cannot alter AMS identity or grant a staff role.

3.3.4 Parsing AMS Webpages Using Python and the DOM

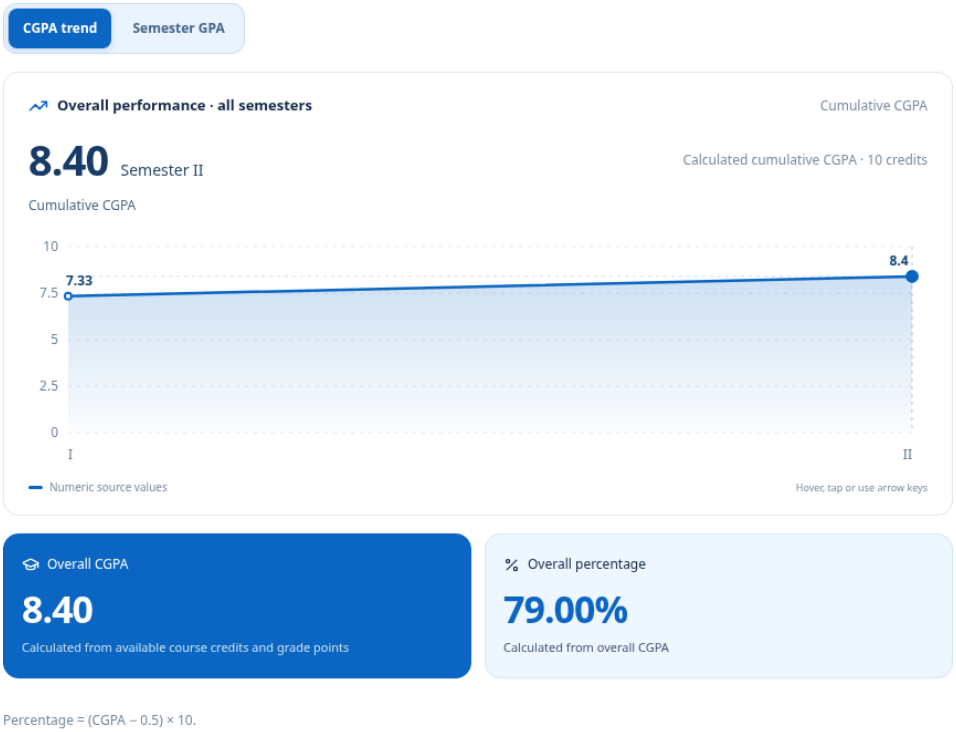


Figure 3.6: Academic records presented in the results dashboard

Figure 3.6 illustrates the academic presentation layer after supported records are normalized. The Python adapter uses verified-TLS requests and structured HTML parsing to read labelled AMS reports. Semester, report and examination-session divisions are preserved. Published summaries take precedence over calculated estimates; eligible course credits and grade points support the cumulative graph. The fictional example displays 8.40 CGPA and 79% using the requested conversion. Unsupported page layouts are marked unavailable, while accepted records retain their original timestamp. Official university submissions are not performed by this parser.

3.3.5 Creation of Webserver Using Node.js

```
if(process.argv[1] && path.resolve(process.argv[1])===fileURLToPath(import.meta.url)) {
  dotenv.config({path:process.env.CAMPUS_ENV_FILE || path.resolve(path.dirname(
      fileURLToPath(import.meta.url)),'.env')});
  const mongoUri=process.env.MONGODB_URI;if(!mongoUri) throw new Error('Set MONGODB_URI
      in campus/.env');
  await mongoose.connect(mongoUri);
  // Enforce uniqueness before accepting registrations or account links.
  await Promise.all([User.init(),Event.init(),Registration.init(),Session.init(),
      AcademicLink.init(),IdentityChallenge.init(),IdentityProof.init(),CampusMedia.init
      (),Follow.init(),AcademicCalendarEntry.init(),TimetableRoomOverride.init(),
      AppFeedback.init(),initializeNotificationModels()]);
  // Other initialization and scheduled jobs omitted from this excerpt.
  const app=createCampusApp({mail});app.listen(Number(process.env.CAMPUS_PORT || 4100),
      process.env.CAMPUS_HOST || '127.0.0.1',()=>console.log('veltech-ams-mini API
      started'));
}
```

Figure 3.7: Node.js campus webserver implementation

Figure 3.7 shows an excerpt from the project's Express server. The service connects to MongoDB, initializes required models and starts the campus application. Its REST routes provide sessions, academic bridging, events, posts, forms, notifications and management. Protected requests enforce ownership, role scope and CSRF checks; request and file limits protect relevant inputs. Private responses use no-store handling. The built React interface can be served by the same campus container, keeping the browser and campus API on a common public origin.

3.3.6 Design of the Three-Tier Application with MongoDB

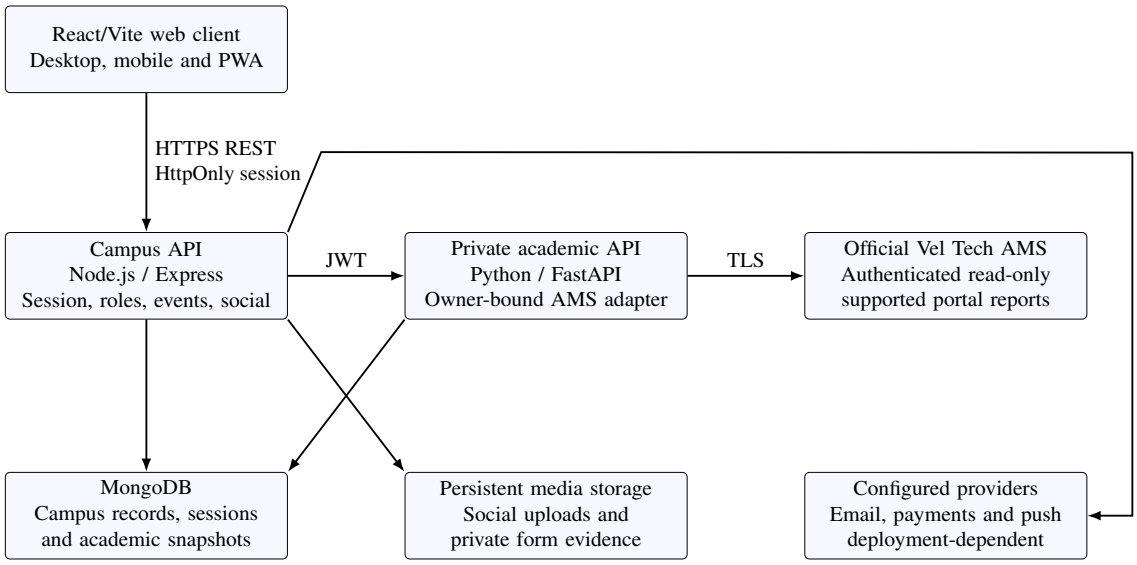


Figure 3.8: Three-tier application and private academic integration

Figure 3.8 shows the project's presentation, application and persistence responsibilities. React forms the presentation tier. Node.js handles campus workflows and bridges owner-bound academic requests to the private FastAPI service. MongoDB stores campus records, sessions and academic snapshots, while persistent media storage holds checked uploads. Official academic data is excluded from public social queries. The AMS connection is a supported authenticated read path; this diagram describes the student project and does not claim to represent the university's internal server architecture.

3.3.7 Design of Reactive Forms in React

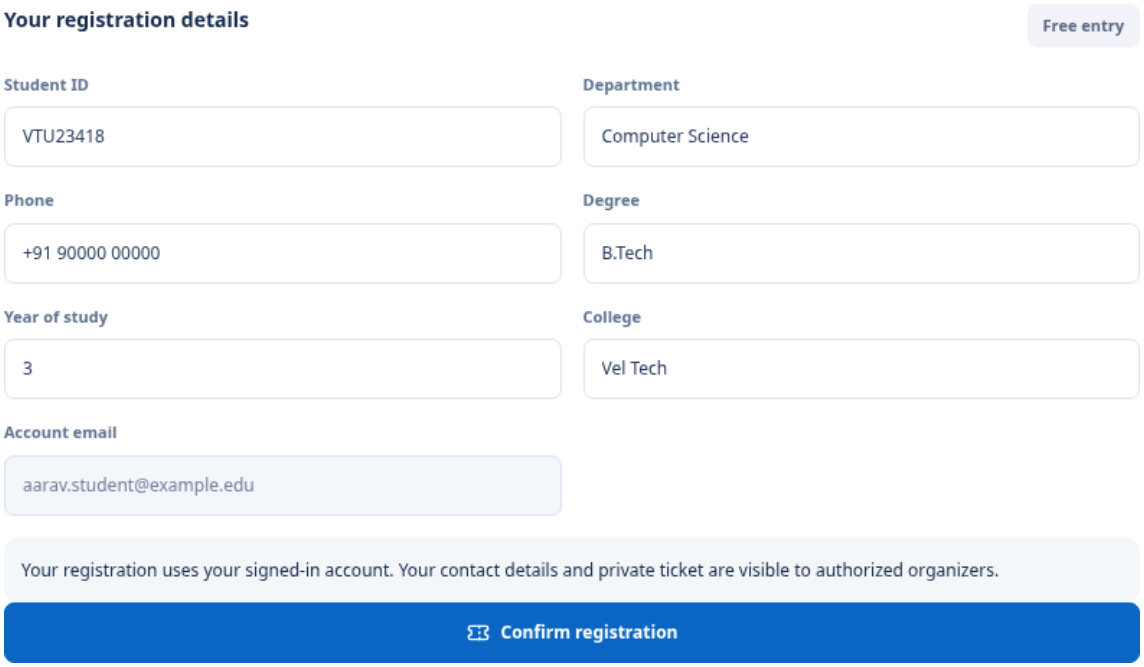


Figure 3.9: Reactive event-registration form

Figure 3.9 shows the controlled event-registration form. Student ID, department, phone, degree, study year and college are collected for the selected event; the account email comes from the signed-in workspace. Input changes update React state and the submit action exposes progress, confirmation or a recoverable error. Free events proceed to registration. Paid events require a configured provider and server-verified payment before issuing a ticket. The displayed event and student values are fictional; the report screenshot does not establish a real payment or event booking.

3.3.8 Routing and Navigation in React

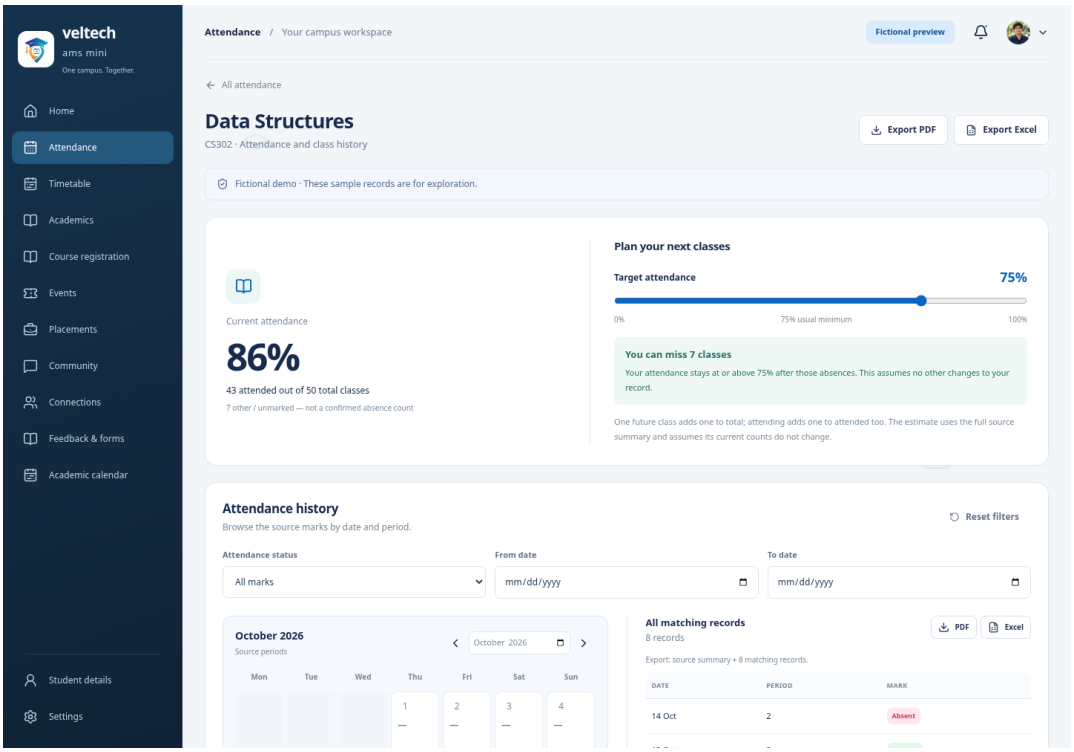


Figure 3.10: React routing to subject attendance

Figure 3.10 shows a dedicated subject-attendance route. Selecting a subject opens its full history, planning information and dated records while preserving the common navigation shell. The calendar and filters narrow the displayed history without changing source totals. Desktop navigation becomes a bottom bar with More on phones. Events, tickets, profiles and forms also use dedicated routes; comments remain associated with the active post or reel. These navigation choices let students move between academic and campus work without losing the relevant context.

3.3.9 Creation of Microservices

```
services:
  academic:
    build:
      dockerfile: docker/academic.Dockerfile
    env_file: .private/academic.env
  campus:
    build:
      dockerfile: docker/campus.Dockerfile
    env_file: .private/campus.env
    volumes:
      - campus-media:/app/media
    depends_on:
      academic:
        condition: service_healthy
```

Figure 3.11: Campus and academic services in Docker Compose

Figure 3.11 shows the service definitions used by the project. The campus API owns social interaction, events and application sessions; the private academic API owns AMS integration. Separate environment files hold private configuration and the media volume survives container recreation. Campus startup waits for the academic health check. Login and report work use bounded admission instead of unlimited parallel source requests. The delivered academic topology uses one worker because pending university verification clients are process-local; multi-host replication requires an explicit session strategy.

3.3.10 Deployment of Microservices

```
RECORDED LOCAL DOCKER BUILD
#21 exporting layers
#21 exporting layers 0.9s done
#21 writing image sha256:977
    d81295bd43f98dbbc62976844268956a34a98486a139824149b25f997ba36 done
#21 naming to docker.io/library/veltech-smtp-qa-campus:latest done

LOCAL SMTP CHECK
OK SMTP email configuration is structurally valid. Credentials are hidden.
NOTE Set CAMPUS_PUBLIC_URL to your public HTTPS origin to include app links in emails.
OK SMTP connection and configured security/authentication checked. No email has been
    sent by this verification step.
OK The provider accepted the requested test message. Check the recipient inbox and Spam
    folder to confirm delivery.
```

Figure 3.12: Recorded Docker build and local SMTP verification

Figure 3.12 shows recorded output from the project's Docker build and a local SMTP verification. The resulting campus image was checked against the final mail source and delivered both HTML and plain text to the local test inbox. Windows setup uses Docker Desktop, PowerShell and Atlas without a separate Ubuntu installation. Public operation additionally needs HTTPS, the correct browser origin, persistent keys/media and real provider configuration. This local evidence is not a claim of public university deployment or Internet inbox delivery.